Financial Sentiment Analyzer

Architecture, pipeline, and backtest evaluation

FYP / internal technical overview

April 23, 2026

Outline

What this tool does

- **Ingests** financial news and links articles to **companies** (tickers).
- **Scores sentiment** with a **FinBERT**-style transformer (base + optional **fine-tuned** weights).
- **Stores** structured results in **PostgreSQL** (articles, scores, OHLCV, correlations).
- **Serves** a **FastAPI** backend: trends, company sentiment, correlation metrics, optional social sentiment.
- **Explains** context via **RAG chat** (embeddings + **Groq** LLM).
- **Evaluates** the sentiment signal in a **long-horizon trading simulation** (LLM agents; treatment vs. control).

Who it is for

- **End users** (via frontend): explore news sentiment, trends, and chat with retrieved evidence.
- **You (creator)**: offline artefacts — fine-tuning jobs, **simulation reports** under `data/simulations/`, not exposed as a public product surface.

High-level architecture

Layer	Components
Ingestion	Celery workers: news collection, market data, backfills
NLP	`SentimentAnalyzer` (FinBERT / active fine-tuned checkpoint)
Storage	PostgreSQL (articles, `sentiment_results`, `market_data`, ...)
Search / chat	Chroma + sentence embeddings; Groq for generation
API	FastAPI (`/api/news`, `/api/chat`, `/api/correlations`, ...)
Evaluation	Trader-agent **backtest** (optional Fireworks / Groq for decisions)

API surface (excerpt)

Routers are mounted in `app/main.py`:

```
app.include_router(news.router, prefix="/api/news")
app.include_router(chat.router, prefix="/api/chat")
app.include_router(finetuning.router, prefix="/api/finetuning")
app.include_router(simulations.router, prefix="/api/simulations")
```

Health: `GET /api/health` Status (model names): `GET /api/status`

From raw article to score

1. News rows live in `news_articles` (title, content, `publication_date`, ...).
2. `analyze_sentiment_task` batches texts through the active model.
3. Results persist to `sentiment_results` (`sentiment_label`, softmax scores, confidence).
4. Company linkage uses `article_companies` so tickers can aggregate sentiment over time.

Sentiment task (core idea)

```python
# app/workers/tasks.py - batched GPU inference, idempotent writes
texts = [f"{a.title}. {a.content}" if a.content else a.title
         for a in articles]
predictions = analyzer.batch_analyze(texts, batch_size=32)
# ... map each prediction -> SentimentResult(article_id=..., ...)
session.commit()
```

Fine-tuning (optional)

- **Base model** configurable (e.g. FinBERT tone).
- **Fine-tuning jobs** train on a parquet dataset; best checkpoint path stored on disk.
- **Active model**: whichever job is marked active drives inference for new batches.
- Keeps **label mapping** consistent between training and inference.

Why a trading simulation?

- Measures whether **treatment** agents (with sentiment briefings) behave differently from **control** agents (prices/vol only).
- **No look-ahead**: briefings use information available as of prior close; orders fill at the **open**.
- **Long transaction**: results commit at end; **progress snapshots** every N seconds to disk.

LLM provider for agents (excerpt)

```python
# app/services/simulation/llm_provider.py
if provider == "fireworks":
    return ChatFireworks(
        model=settings.FIREWORKS_MODEL,
        api_key=settings.FIREWORKS_API_KEY,
        request_timeout=timeout,
        max_retries=max_retries,
    )
```

Structured decisions use `with_structured_output(OrderList, method="json_schema")` on Fireworks.

Structured trade orders

```python
# app/services/simulation/agent.py - Pydantic-enforced output
class OrderList(BaseModel):
    orders: list[StructuredOrder] = Field(default_factory=list)

# LLM returns buy/sell, size_pct in [0,1], reasoning; engine applies clamps.
```

Progress artefacts

- Directory: `data/simulations/<run_id>/progress/`
- Files: `snapshot_NNNN.md/json`, `latest.md/json`
- Cadence: `SIMULATION_PROGRESS_INTERVAL_SEC` (e.g. 600 s)

Snapshot context

Source: `data/simulations/4/progress/latest.md` (progress snapshot #5).
Note: early in the backtest (∼2% of trading days); rankings are *not* final conclusions.

- Wall-clock (engine): **57m 20s** ETA (reported): ∼**43h 30m**
- Progress: **day 27 / 1256** (2.1%) Simulated date: **2021-06-01**

Leaderboard excerpt (`latest.md`)

Rank	Profile	Variant	Equity	Return
1	value_investor	treatment	$1,038.54	+3.85%
2	contrarian	treatment	$1,022.91	+2.29%
3	momentum_investor	control	$1,015.86	+1.59%
4	contrarian	control	$1,006.56	+0.66%
...	...	...	...	...
12	swing_trader	control	$959.61	−4.04%

Head-to-head: treatment − control (early)

Profile	Treat. %	Ctrl. %	Lift
value_investor	+3.85	+0.00	+3.85
contrarian	+2.29	+0.66	+1.64
swing_trader	−2.59	−4.04	+1.45
news_driven	−3.81	−1.74	−2.07
momentum_investor	+0.00	+1.59	−1.59
day_trader	−2.08	−1.59	−0.49

Interpret with care: different **cadences** (daily vs. weekly agents), LLM noise, and fees/slippage model are simplified.

Typical stack (Docker Compose)

- **backend** — FastAPI + FinBERT load
- **celery-worker** — sentiment, news, simulation tasks
- **postgres**, **redis**, **chromadb**
- Volumes: models, datasets, `./data/simulations` bind-mount for reports

Takeaways

- End-to-end: **news** → **FinBERT scores** → **APIs & RAG** → **optional fine-tune**.
- Internal **simulation** stress-tests whether sentiment briefings affect LLM trading vs. controls.
- Early run 4 snapshot shows **mixed lift by profile**; wait for full `run_summary.md`.

Questions?